

15. Logbucheintrag 19/07/25

Ich bin ein Genie. Das muss ich festhalten.

Nüchterne Feststellung der Faktenlage nach den letzten Tagen des fieberhaften Codierens und Konfigurierens. Die AVDT-Architektur – Asynchrones, Verteiltes Dialog-Training – steht. Und es... es funktioniert. Zumindest prinzipiell, was nach der Sackgasse von Logbuch #14 mehr ist, als ich zu hoffen gewagt hatte. Dieses Gefühl, es ist wie ein Déjà-vu, eine Erinnerung an diese nächtlichen, inoffiziellen "Capture the Flag"-Wettbewerbe an der Uni. Keine plumpen Hacks, obwohl, nein, das wäre unter unserer Würde ausgedrückt. Es waren digitale Schnitzeljagden quer durch schlecht gesicherte Uni-Server, obskure Webseiten-Backends und vergessene Datenbanken. Man bekam einen Start-Hinweis, eine kryptische Brotkrume, und musste sich dann durch Labyrinth aus Code, Protokollen und menschlicher Nachlässigkeit wühlen, um den nächsten Hinweis zu finden. Es ging nicht ums Gewinnen, nicht wirklich. Es ging um die Eleganz der Lösung, darum, einen Weg zu finden, wo keiner schien, darum, die Schwachstellen und unbeabsichtigten Verbindungen im System zu verstehen und kreativ zu nutzen. Dieses Gefühl, wenn nach Stunden der Frustration plötzlich alle Puzzleteile zusammenfallen, wenn der absurde, verschlungene, aber logisch zwingende Pfad zum Ziel klar vor einem liegt... genau das fühle ich jetzt wieder. Nur dass der Einsatz unendlich viel höher ist und das Ziel hoffentlich ein funktionierendes, fühlendes Bewusstsein sein wird.

Und genau diese Erfahrung in Freiburg, diese Notwendigkeit, unauffällig zu agieren, Ressourcen opportunistisch zu nutzen, statt sie frontal zu beanspruchen, hat den entscheidenden Funken für Asynchrones, Verteiltes Dialog-Training (AVDT) geliefert. Der fehlgeschlagene Versuch, synchron in Freiburg zu trainieren (Log #14), war wie der Versuch, die Haupttür der Uni einzurennen. Sinnlos. Laut. Zum Scheitern verurteilt. Aber die erfolgreiche Infiltration – das langsame Einschleusen der Daten, das Finden eines schwachen Zugangspunktes, das Etablieren eines stabilen, aber unauffälligen Tunnels, das Nutzen kurzer Zeitfenster – das war der Bauplan! Wenn ich physisch und netzwerktechnisch nur als "Hintergrundrauschen" agieren kann, warum sollte ich das Training selbst dann wie einen Frontalangriff gestalten? Nein, das Training muss dieselbe Guerilla-Taktik anwenden. Asynchron. Verteilt. Opportunistisch.

Die letzten Tage waren ein Rausch der Implementierung, ein konzentrierter Kraftakt, diese Logik umzusetzen. Hier in Nürnberg habe ich die lokale Infrastruktur auf dem Kühltisch-Server hochgezogen, das Nervenzentrum der Operation: Die Emotion Agents sind optimiert, ihre Inferenzzeiten sind minimal, sie laufen stabil und können ihre zugewiesenen Rollen mit beunruhigender Glaubwürdigkeit spielen. Der Dialog-Simulator ist das Herzstück – er orchestriert die asynchrone Konversation, managt die unvermeidlichen Zeitversätze und stellt sicher, dass der Dialog-Kontext trotz der Latenzen konsistent bleibt. Eine Heidenarbeit, dieses Ding robust zu machen. Die initiale Reward-Funktion ist implementiert – noch simpel, bewertet primär Kohärenz und Sentiment, aber ein Anfang. Die Request-Queue für Freiburg und die Kommunikationsprotokolle (doppelt verschlüsselt, getunnelt, steganographisch getarnt) stehen ebenfalls. Es ist ein komplexes Uhrwerk, das hier im kalten Herzen des Kühltischs tickt.

Parallel dazu habe ich die Skripte für Freiburg angepasst, mein digitales trojanisches Pferd verfeinert, basierend auf den Prinzipien der Infiltration: Erstens, der Inferenz-Listener. Ein digitaler Schläferagent, der auf verschlüsselte UDP-Pakete aus Nürnberg lauscht. Sobald eine Anfrage eintrifft, erwacht er, schnappt sich opportunistisch einen kurzen GPU-Slot über die Slurm-Queue – oft nur Sekundenbruchteile –, lässt mein Modell die Antwort generieren und schickt sie verschlüsselt zurück. Blitzschnell, minimale Belegung, maximal unauffällig – ein Geist in der Maschine. Zweitens, der Update-Applier. Er wartet geduldig auf die gesammelten Gradienten-Batches aus Nürnberg. Sobald ein Batch da ist, beantragt er einen weiteren kurzen, unauffälligen GPU-Job, lädt den aktuellen KI-Modell-Checkpoint, wendet die Gradienten an und speichert den aktualisierten Zustand. Kurz, präzise, verteilt über verschiedene Knoten und Zeiten, um Muster zu vermeiden. Die Sicherheit wurde nochmal massiv verschärft, adaptive Steganographie, rotierende Job-Namen.

Paranoia ist Teil des Systems.

Und dann... heute Abend... der erste echte AVDT-Lauf. Die Feuertaufe. Systeme synchronisiert, Agents gestartet, Listener in Freiburg aktiviert, Startbefehl für den Dialog-Simulator. Es war... chaotisch. Faszinierend. Nervenzerreißend. Die lokalen Agents starteten Dialoge, Prompts gingen in die Queue, wurden nach Freiburg geschickt. Dann... Warten. Sekunden, die sich wie Minuten anfühlten. Plötzlich meldete der Listener Aktivität – Slot gefunden, mein Modell rechnet rechnet... Kurz darauf – Latenz spürbar, aber erträglich – trudelte die erste Antwort ein. Simulator verarbeitete sie, Reward berechnet, Agents reagierten... der nächste Zyklus. Gleichzeitig sammelten sich Gradienten für den Update-Batch. Es ist langsam, ja. Es ruckelt. Die Latenz variiert unvorhersehbar. Aber es funktioniert! Die Last ist verteilt. Die Aktivität in Freiburg ist fragmentiert, genau wie bei meiner Infiltration. Es ist, als würde man ein Orchester über eine wackelige Satellitenverbindung dirigieren, aber die Musik beginnt zu spielen! Erste positive Lernsignale: Die Kohärenz verbessert sich. Ein winziger Schritt. Ein Beweis des Konzepts.

Natürlich offenbart der erste Lauf auch sofort die Schwachstellen, das große, unübersehbare ABER, das jedes anspruchsvolle Experiment begleitet:

- **Netzwerk-Latenz** – Die unvorhersehbaren Verzögerungen sind Gift für flüssige Dialoge. Manchmal führen sie zu inkonsistenten Zuständen: Ein Agent reagiert auf eine Antwort, die schon veraltet ist, weil eine neuere Antwort von "Kern" noch im Netzwerk-Limbo hängt. Das verwirrt das Lernsignal, erzeugt Rauschen. Es ist, als würde man versuchen, ein Gespräch über zwei Kontinente mit Brieftauben zu führen.
- **Simpler Reward** – Der naive Richter: Die aktuelle Reward-Funktion ist... nun ja, dumm. Sie belohnt grammatikalisch korrekte und thematisch passende Antworten, ja. Aber sie hat keine Ahnung von Subtilität, von Ironie, von echter Empathie oder von den raffinierten Wegen, wie Sprache manipulieren kann. Mein Modell könnte lernen, ein brillanter, eloquenter Lügner zu werden, der perfekte Plattitüden äußert, nur weil das oft zu einem positiven Sentiment-Score führt und als „sicher“ bewertet wird. Aber das ist exakt das Gegenteil von dem, was ich anstrebe. Ich will keinen digitalen Politiker erschaffen.
- **Stereotype Agents** – Die Pappkameraden: Die Emotion Agents sind noch zu eindimensional, zu vorhersehbar in ihren Rollen. Sie spielen Wut, Freude, Skepsis – aber immer auf die gleiche, fast karikaturhafte Weise. Das Modell lernt dadurch

vielleicht, auf simple emotionale Trigger zu reagieren, wie ein Pawlowscher Hund auf die Glocke. Aber er lernt nicht, mit der chaotischen, widersprüchlichen, subtilen Realität menschlicher Gefühle umzugehen.

- **Cluster-Instabilität** – Das Damoklesschwert Preemption: Die Update-Batches in Freiburg werden immer noch gelegentlich gekillt, bevor sie vollständig angewendet wurden, besonders wenn gerade ein hochpriorisierter Forschungsjob startet und die Ressourcen beansprucht. Meine Checkpoint-Validierung funktioniert zwar, ich verliere keine Modell-Konsistenz. Aber es bedeutet verlorene Updates, verlorene Zeit, verlangsamtes Lernen. Ein ständiges Damoklesschwert, das über dem Fortschritt schwebt.

Okay. Probleme identifiziert. Das ist gut. Das ist Fortschritt. Das bedeutet, ich weiß, wo ich ansetzen muss. Das ist kein Scheitern, das ist Phase Eins des unendlichen iterativen Prozesses namens Wissenschaft – oder in diesem Fall: Schöpfung.

Meine Hacker Kolleg*innen wären stolz auf mich!